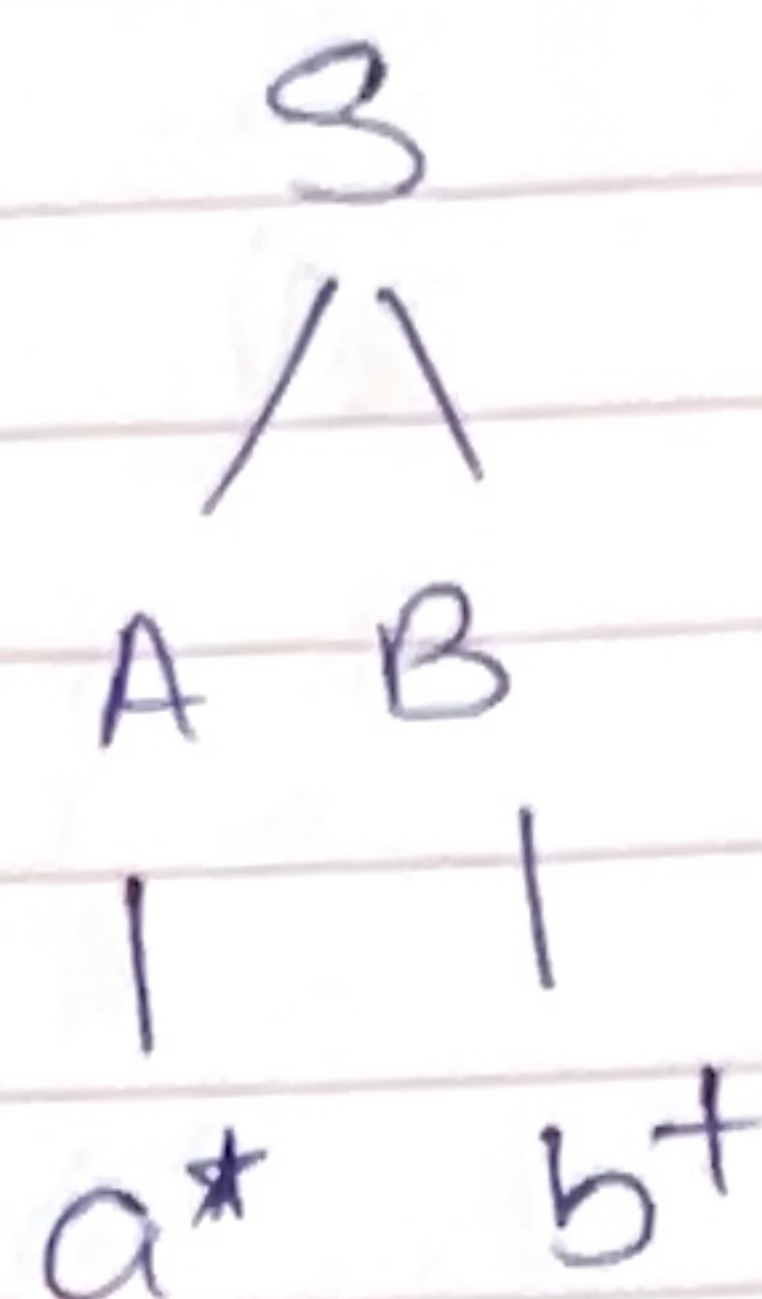
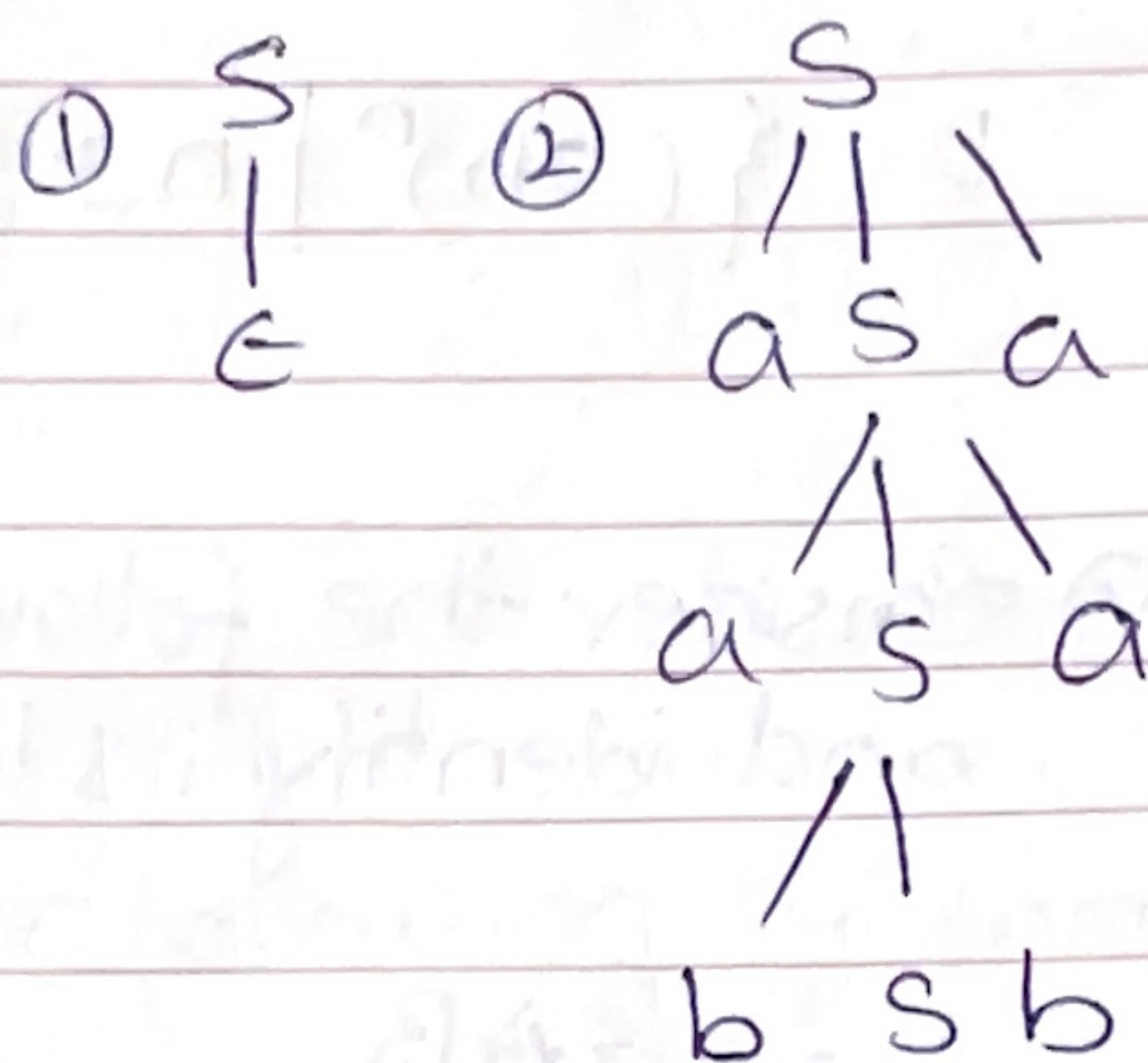


(vi) $S \rightarrow AB$ $A \rightarrow aA / \epsilon$ $B \rightarrow bB / b$ 

$$L = \{a^n b^m \mid n \geq 0 \text{ \& } m \geq 1\}.$$

(vii) $S \rightarrow aSa / bSb / \epsilon$  $\Rightarrow$ outputaabbba!

a palindrome.

CHOMSKY CLASSIFICATION OF LANGUAGES:-

Chomsky classified the grammar into four types in terms of production (type 0-3)

This hierarchy of grammar was described by Noam Chomsky in 1956. From type 0 — 3 we will be putting more and more restrictions.

Type-0 GRAMMAR:- $\rightarrow$ (one that we've practised till now)

A grammar without any restrictions.

But this would result in much complex languages generated by the grammar. Such grammar is thus handled through a turning machine.

TYPE 1 GRAMMAR:-

Also known as case sensitive grammar,
length increasing grammar.

This is used to generate case sensitive language
which is accepted by linear Bound automaton.
(LBA).

Description

$$\alpha \rightarrow \beta$$

$$\alpha \in \{\Sigma \cup V_n\}^* V_n \{\Sigma \cup V_n\}^*$$

$$\beta \in \{\Sigma \cup V_n\}^*$$

} same as what we've
been studying
before.

But; $|\alpha| < |\beta|$ length of alpha should be less than
equal to length of Beta!

e.g. ① $ABC \rightarrow ab$

↳ not acceptable.

② $S \rightarrow \epsilon$

↳ not acceptable as from the rule but
it is accepted as an exception.

TYPE - 2 GRAMMAR:- (all the analysis by computer,
like parsing is done through this).

Also known as context free grammar which will
generate context free language that will be accepted
by Push down automata.

Description

$$\alpha \rightarrow \beta$$

$$\alpha \in V_n \quad |\alpha| = 1$$

$$\beta \in \{\Sigma \cup V_n\}^*$$

L.H.S has no
right or left
context.



TYPE - 3 GRAMMAR.

Used to generate Regular Grammar which will generate regular language which will be accepted by finite machine.

Regular Grammar can be of 2 types.

↓
left linear

↓
Right linear.

$$A \rightarrow a/Ba.$$

$$A, B \in V_n. \quad |A| = |B| = 1.$$

$$a \in \Sigma^*$$

$$A \rightarrow a/aB$$

$$A, B \in V_n$$

$$a \in \Sigma^*$$

However if left linear rules and right linear rules are combined the language is no longer be regular as it does not follow any path $\rightarrow$ right/left.

Question: Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$ where every accepted string starts with substring $s = abb$.

we want something given in R.E as $\overbrace{abb}^A (\overbrace{a+b}^B)^*$

let.

$$S \rightarrow AB$$

$$A \rightarrow abb$$

$$B \rightarrow aB/bB/\epsilon$$

} grammar.

Question: ... such that every accepted string starts and ends with A / a.

What we want can be represented through regular expressions as.

$$a + a(atb)^*a.$$

let

$$\frac{a}{A} + \frac{a}{A} \frac{(atb)^*}{B} \frac{a}{B}$$

$$S \rightarrow A / ABA$$

$$A \rightarrow a$$

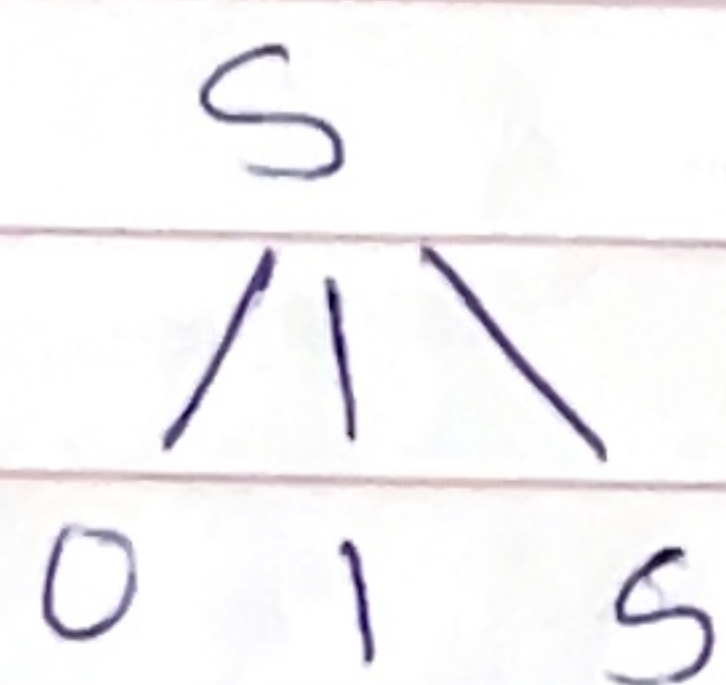
$$B \rightarrow aB / bB / \epsilon$$

} required grammar Answer!

Regular Grammar to Regular Expression.

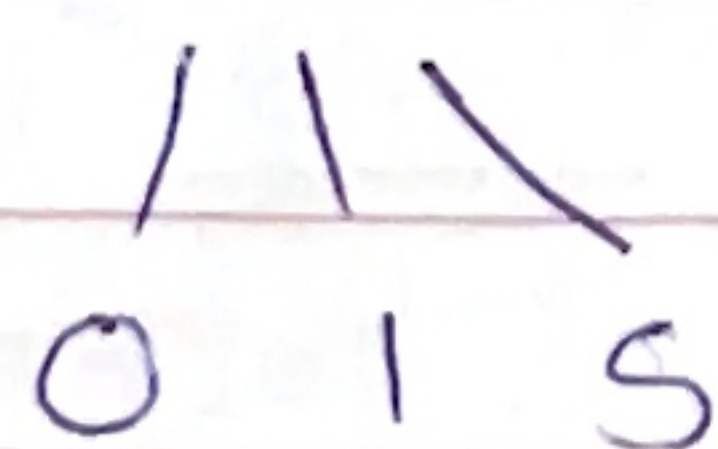
$$\textcircled{1} S \rightarrow 01S / 01$$

Step no. 1 Draw tree diagram.

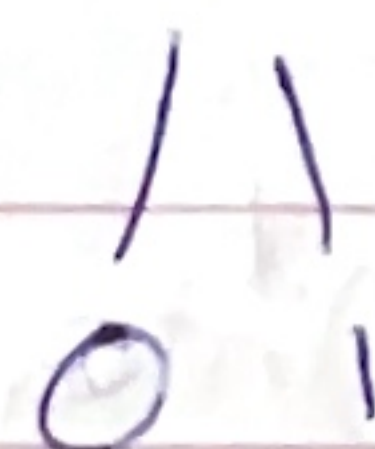


step no. 2

Deduce pattern.



RE: $(01)^+$ Ans.



$$\textcircled{ii} S \rightarrow 0011S / 01 / 10.$$

Step no. 1: Similar.

Step no. 2: $(0011)^* (01 + 10)$

$$\textcircled{iii} S \rightarrow 01A / B11$$

$$A \rightarrow 011A / 01$$

$$B \rightarrow 101B / 11$$

Step no. 1 lets draw R.E expressions for A & B.
(you may use trees if needed.)

$$A \rightarrow 011A / 01 \Rightarrow (011)^* (01)$$

$$B \rightarrow 101B / 11 \Rightarrow (101)^* (11)$$

$$S \rightarrow 01A / B11 \quad \text{placing values of A \& B.}$$

$$\Rightarrow 01[(011)^* (01)] / [(101)^* (11)] 11$$

$$01(011)^* 01 + (101)^* (11)(11) \text{ Ans!}$$

$$\textcircled{iv} S \rightarrow 011A / 101B \quad \text{Important!}$$

$$A \rightarrow 110A / 00$$

$$B \rightarrow 11B / S.$$

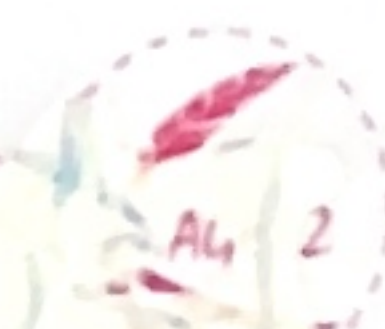
$$A \rightarrow 110A / 00 \rightarrow (110)^* 00 \quad \text{--- (a)}$$

$$B \rightarrow 11B / S \rightarrow (11)^* S$$

$$\text{let } S \rightarrow 011(110)^* 00 / 101(11)^* S$$

S can repeat as many time as one want

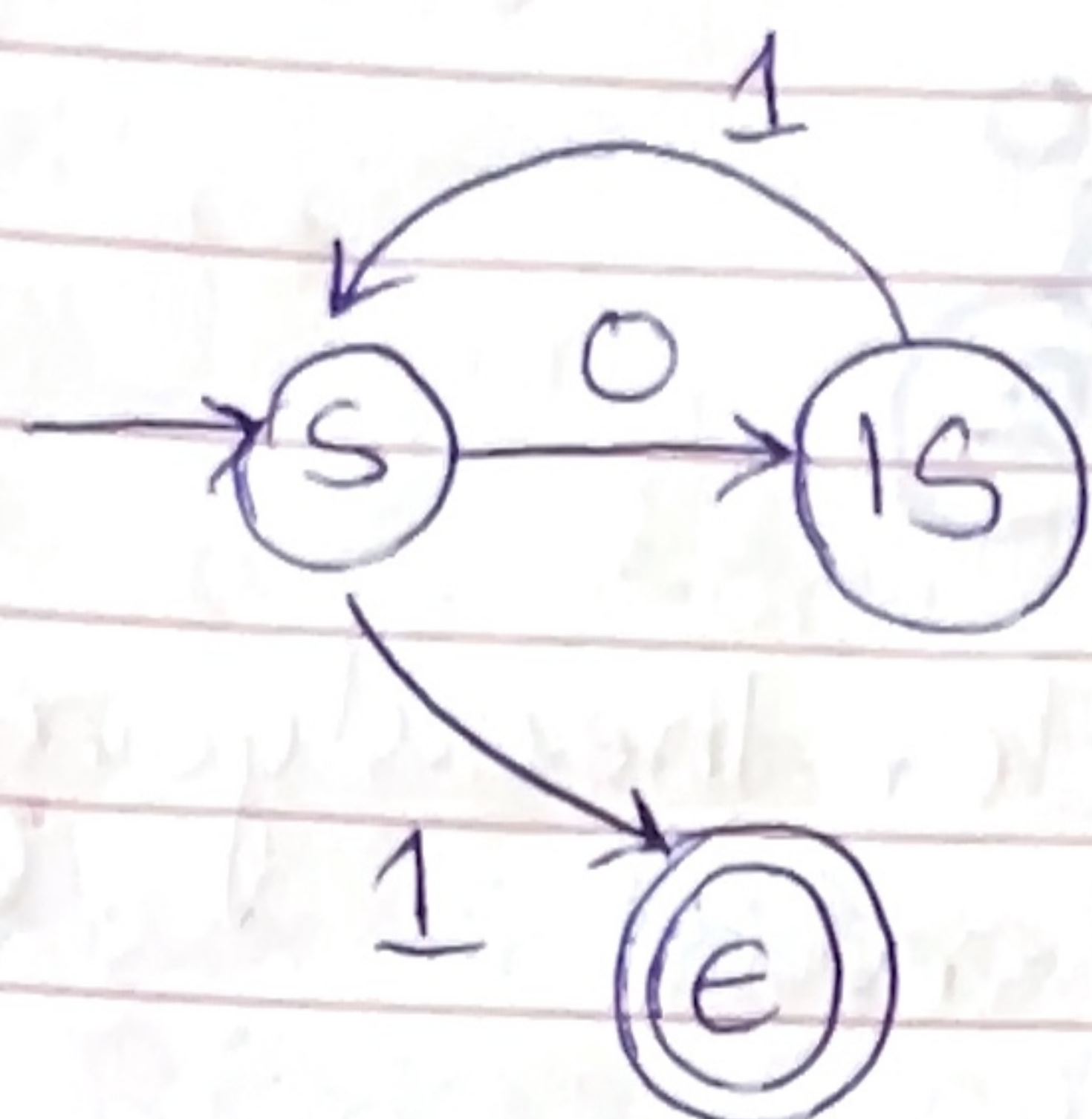
$$101(11)^* S 011(110)^* 00 \text{ Ans!}$$



Specifically right regular (left and right are equivalent and interchangeable by a proper method).
 Regular Grammar to Finite Automata.

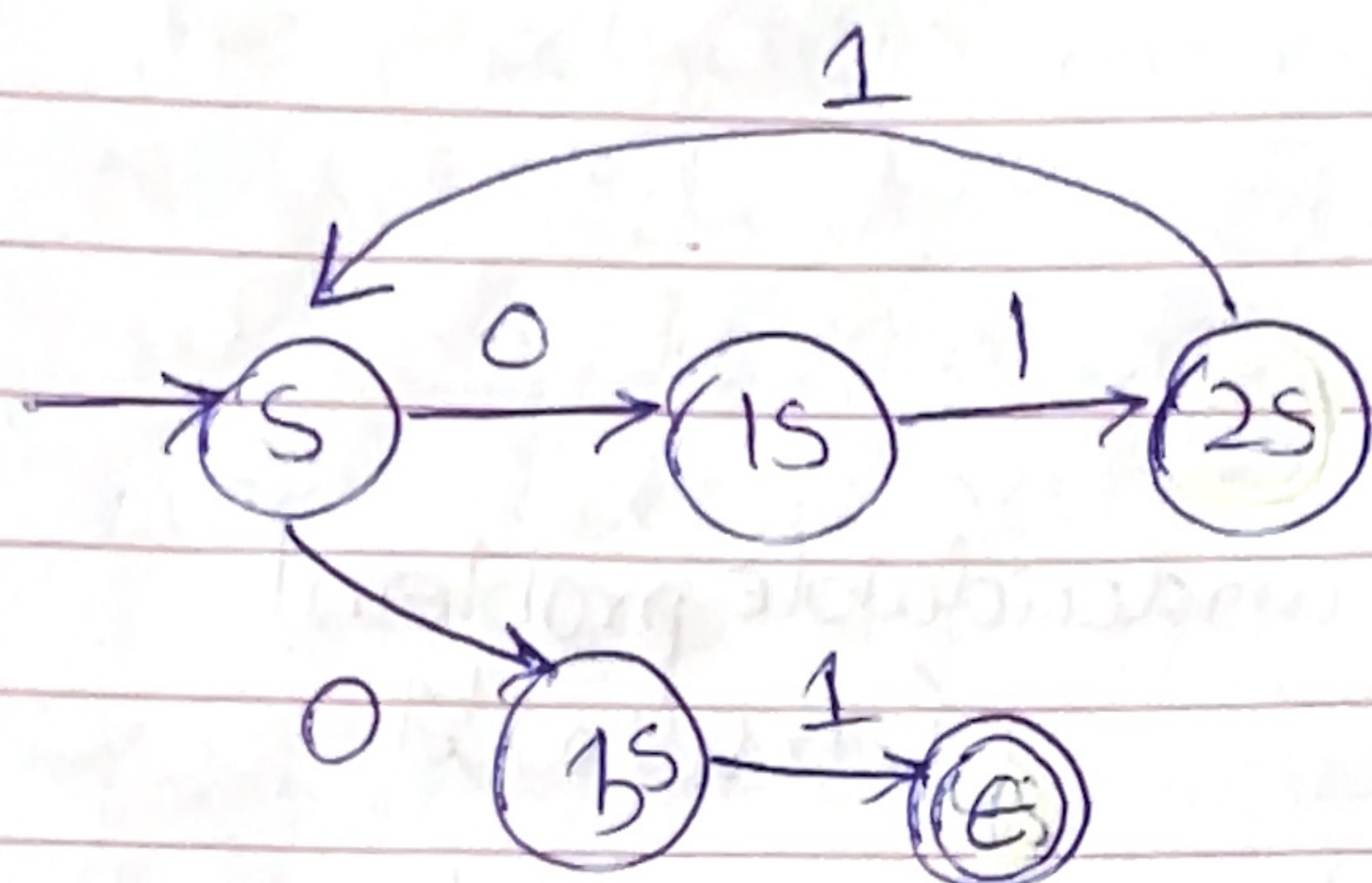
① $S \rightarrow 01S / 1 \rightarrow R.E (01)^*1.$

proper method.



② $S \rightarrow 011S / 01$ Imp!

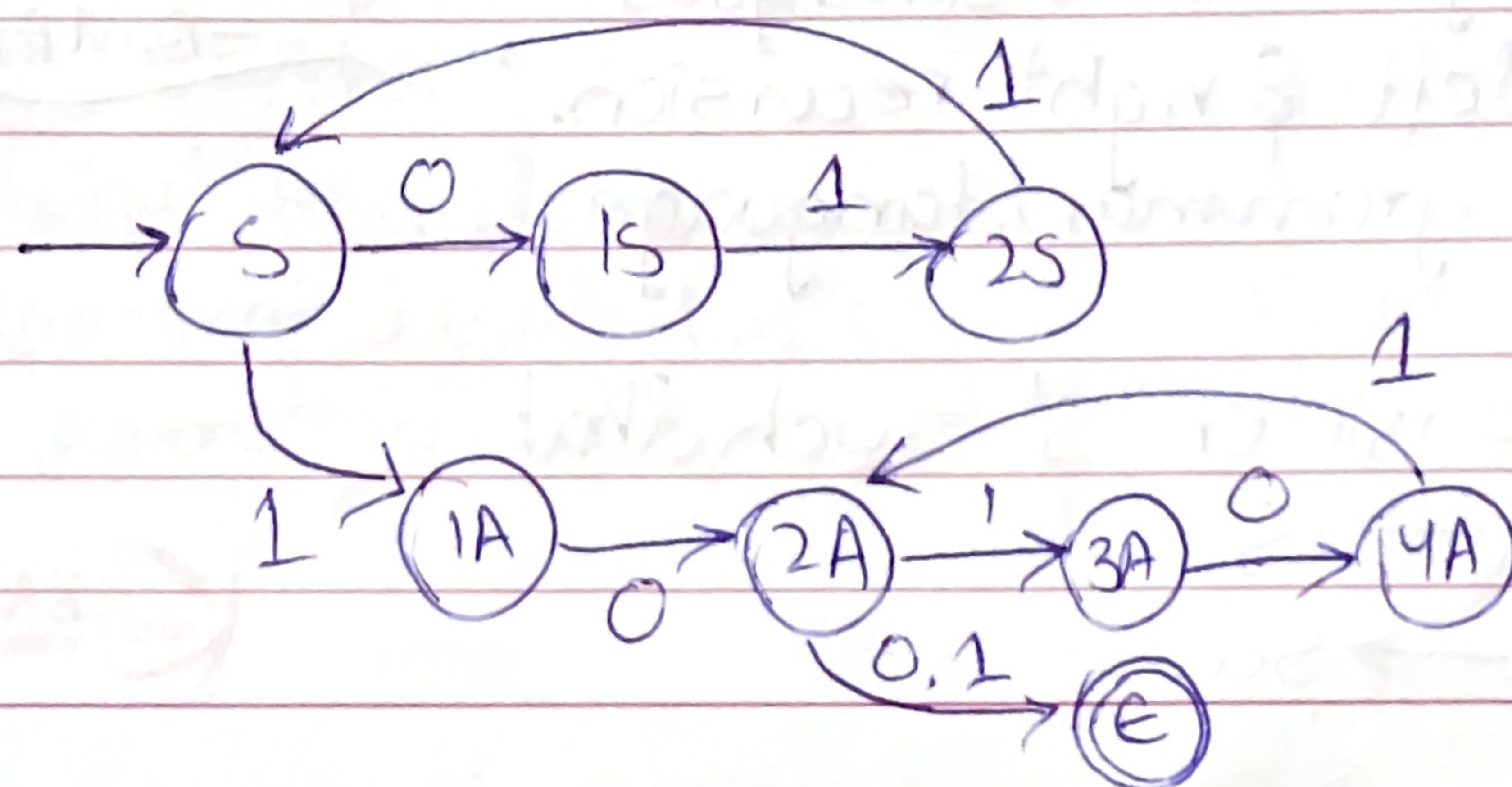
$(011)^*01$



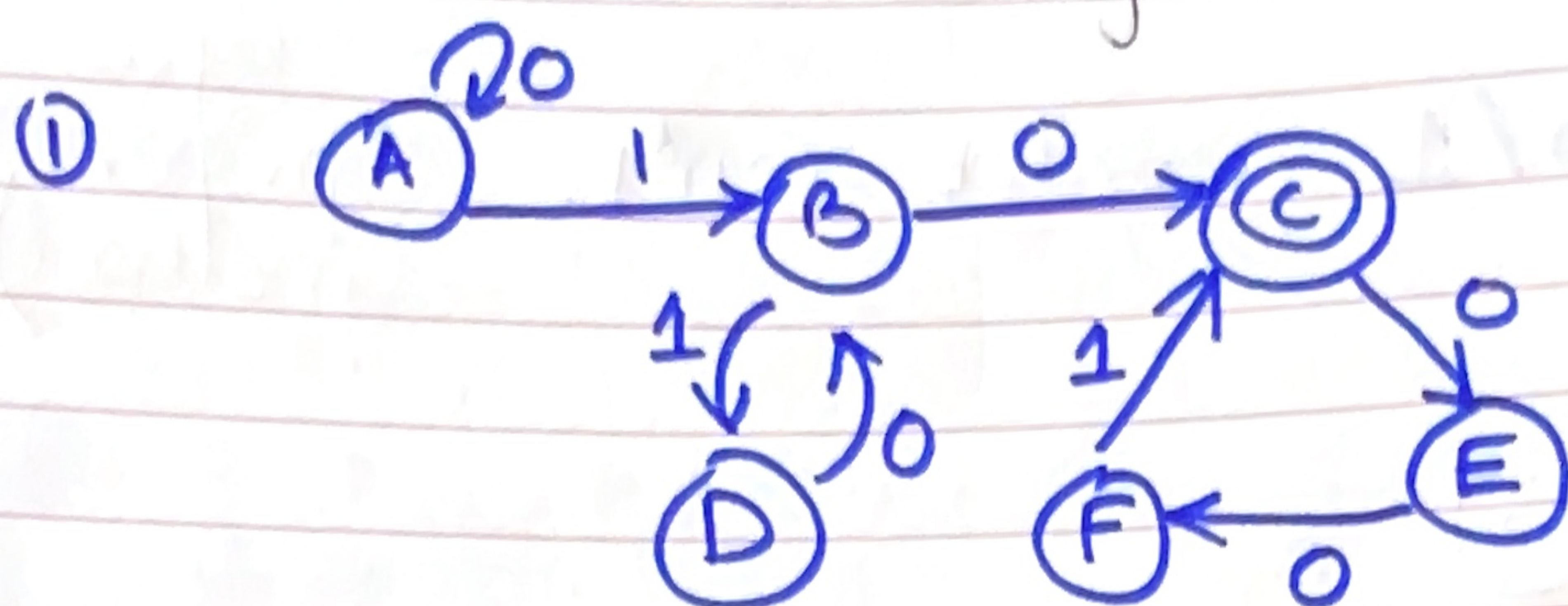
③ $S \rightarrow 001S / 10A$ Imp!
 $A \rightarrow 101A / 0 / 1$

$A \rightarrow (101)^*(0+1)$

$S \rightarrow (001)^*(10(101)^*(0+1))$



Finite Automata to Regular Grammar-



trick: Only consider the outgoing path and enlist accordingly.

$A \rightarrow 0A / 1B$

$B \rightarrow 1D / 0C$

$C \rightarrow 0E / \epsilon$

$D \rightarrow 0B$

$E \rightarrow 0F /$

$F \rightarrow 1C$

Ambiguous Grammar:- $\rightarrow$ undecidable problem!
 CBT & LBO i/t

The grammar CFG is said to be ambiguous if there are more than one derivation tree for any string if there exists more than one derivation. $\downarrow$ tree. (LMDT / RMDT).
 (ie)

Example $\Rightarrow S \rightarrow as / sa / a$.

how to find if a grammar is ambiguous

* if / check for left & right recursion.
 in the same grammar / language.

x } some languages may have only ambiguous CF grammar
 — Rohit Parikh 1961

example: Consider 'S' such that

$S \rightarrow aS$

$S \rightarrow Sa$.

SIMPLIFICATION / MINIMIZATION OF CFG.

The reason we simplify a CFG is to make it more efficient and compiler friendly. The process of deleting and eliminating of useless symbols^① unit productions and^② Null productions^③, is known as CFG.

→ order of operation does not matter tho.

(1) REMOVAL OF NULL OR EMPTY PRODUCTIONS.

The production from $A \rightarrow \epsilon$ is known as the NULL production or empty production; here we try to remove them by replacing with derivation that are equivalent.

Consider the following example.

$$\begin{aligned} \textcircled{1} \quad S &\rightarrow AbB \\ A &\rightarrow a / \epsilon \\ B &\rightarrow b / \epsilon \end{aligned}$$

→ remove ϵ and add these dependant productions separately.

$$\begin{aligned} S &\rightarrow AbB / Ab / bB / b \\ A &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

(2) What if the language grammar itself has ϵ as a word.

$$\begin{aligned} \textcircled{2} \quad S &\xrightarrow{\text{CNFD}} AB / b / \epsilon \\ A &\rightarrow a / \epsilon \\ B &\rightarrow b / \epsilon \end{aligned}$$

here we do the following process!

$$\begin{aligned} S' &\rightarrow S \cup \epsilon \\ S &\rightarrow AB / A / B \\ A &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

$$(3) S \rightarrow aSb / \epsilon$$

$$S' \rightarrow S \cup \epsilon$$

$$S \rightarrow aSb / ab$$

} Ans!

II. Removal of Unit Productions.

Say you're given a production/grammar

$$A \rightarrow B$$

$$B \rightarrow C$$

$$C \rightarrow D$$

$$D \rightarrow a$$

$$\Rightarrow A \rightarrow B/a$$

for all single terminal
and single non terminal
we should directly put in
a value.

$\therefore$ The production of the form $A \rightarrow B$ where $A, B \in V_n$, $|A| = |B| = 1$ is known as unit production.

Consider example.

$$(1) S \rightarrow Aa$$

$$A \rightarrow a/B$$

$$B \rightarrow d.$$

$$S \rightarrow Aa$$

$$A \rightarrow a/d$$

$$(2) S \rightarrow aAb$$

$$A \rightarrow B/a$$

$$B \rightarrow C/b$$

$$C \rightarrow D/c$$

$$D \rightarrow d.$$

$$S \rightarrow aAb$$

$$A \rightarrow B/a$$

$$B \rightarrow C/b$$

$$C \rightarrow d/c$$

$$S \rightarrow aAb$$

$$A \rightarrow B/a$$

$$B \rightarrow b/d/c$$

$$S \rightarrow aAb$$

$$A \rightarrow a/b/d/c$$



III. Removal of Useless Symbols

The variable which are not involved in the derivation of any string are known as useless symbols.

- Select the variable that cannot be reached from the start symbol of the grammar and remove them along with their all production.
- select the variable that are reachable from the start symbol but cannot derive any terminal, remove them along with

EXAMPLE

$S \rightarrow aAB$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow d$

→ this is to be eliminated.

→ just like unreachable state of FA.

$S \rightarrow aA(B)$
 $A \rightarrow b$

→ after this we cannot reach any terminal.

→ this is to be eliminated

→ e.g. of a dead state.

CHOMSKY NORMAL FORM.

The Grammar G is said to be in Chomsky Normal If every production is in the form of.

$$A \rightarrow BC / a$$

$\nearrow$ only 2 nonterminals.
 $\nwarrow$ or $\downarrow$ one terminal
 $BC \in V_n$
 $a \in \Sigma$

process of Normalization. this helps to make the given grammar user compiler friendly.

example.

① $S \rightarrow aSb / ab.$

let

$$\begin{array}{lcl}
 S \rightarrow ASB / AB & & S \rightarrow SAB / AB \quad \text{Ans!} \\
 A \rightarrow a & \Rightarrow & SA \rightarrow AS \\
 B \rightarrow b & & A \rightarrow a \\
 & & B \rightarrow b
 \end{array}$$

② $S \rightarrow aAb / bB.$

$$\begin{array}{lcl}
 A \rightarrow a/b & \Rightarrow & S \rightarrow aAB / BB \\
 B \rightarrow b. & & C \rightarrow a \\
 & & A \rightarrow a/b \\
 & & B \rightarrow b
 \end{array}$$

then finally

$\therefore S \rightarrow CAB / BB$

$$\begin{array}{lcl}
 S \rightarrow SAB / BB & & \\
 C \rightarrow a & B \rightarrow b & \text{Ans!} \\
 A \rightarrow a/b & SA \rightarrow CA &
 \end{array}$$

Thus right now we just made a form of having only 2 non-terminals.

Note:-

If a CFG is in CNF, then for a derivation of string 'w' with length we need exactly $2n-1$ production. $|w| = n$, number of Sentential form / steps of derivation will be $2n-1$.

Chapter No. 4: "Push Down Automata".

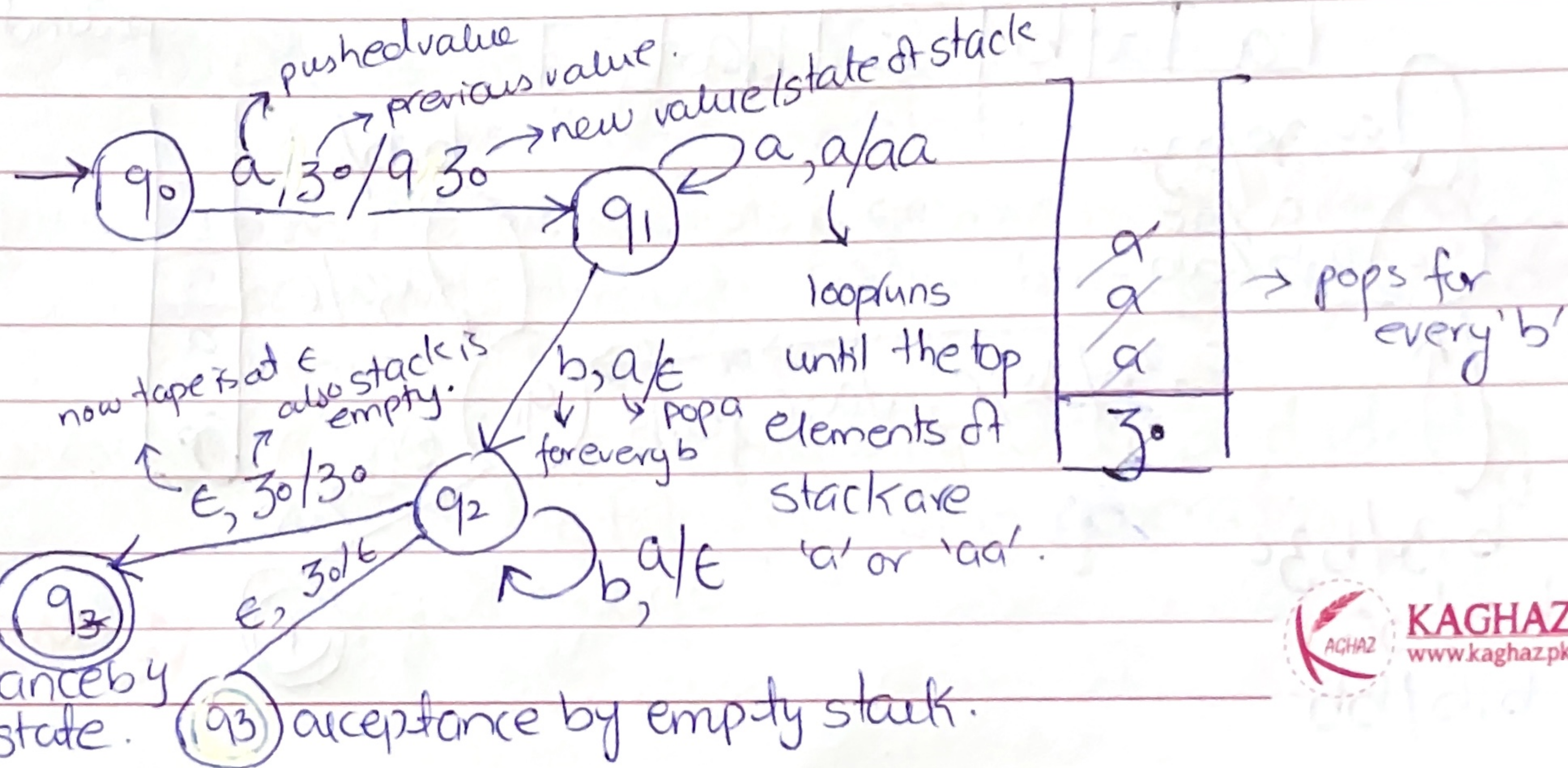
Q. Design a Deterministic Push Down Automata for $\{a^n b^n \mid n \geq 1\}$?

let

a	a	a	b	b	b	ε	ε
---	---	---	---	---	---	---	---

Now we want equal no. of a's and b's. The general idea is to push every 'a' into the stack and then pop every a when a 'b' appears.

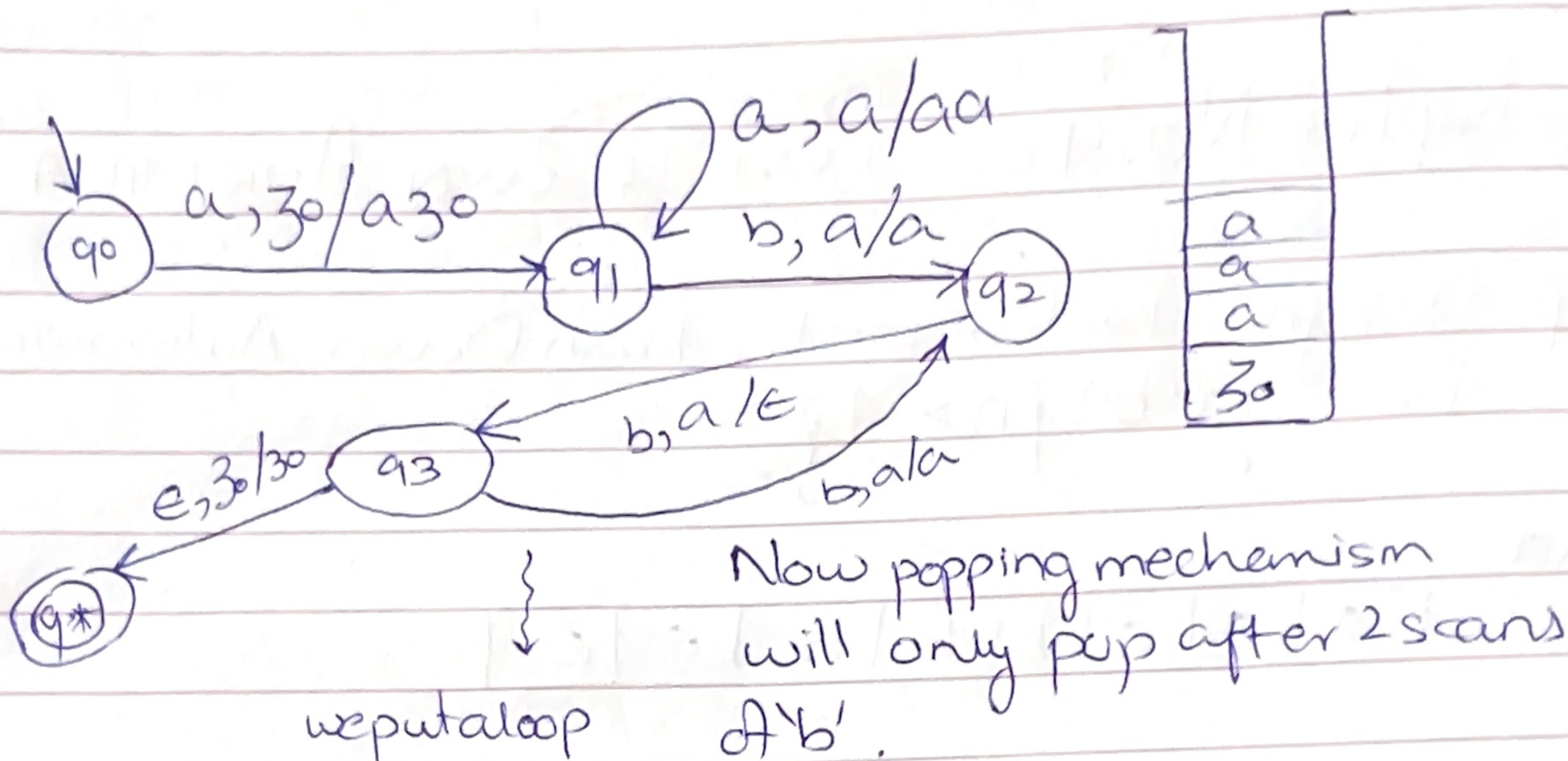
If everything goes planned i.e no. of a's = no. of b's the stack would (as) be empty at the end.



Question. Design a Deterministic Push Down Automata for $\{ a^n b^{2n} \mid n \geq 1 \}$?

let

a a a b b b b b b e e



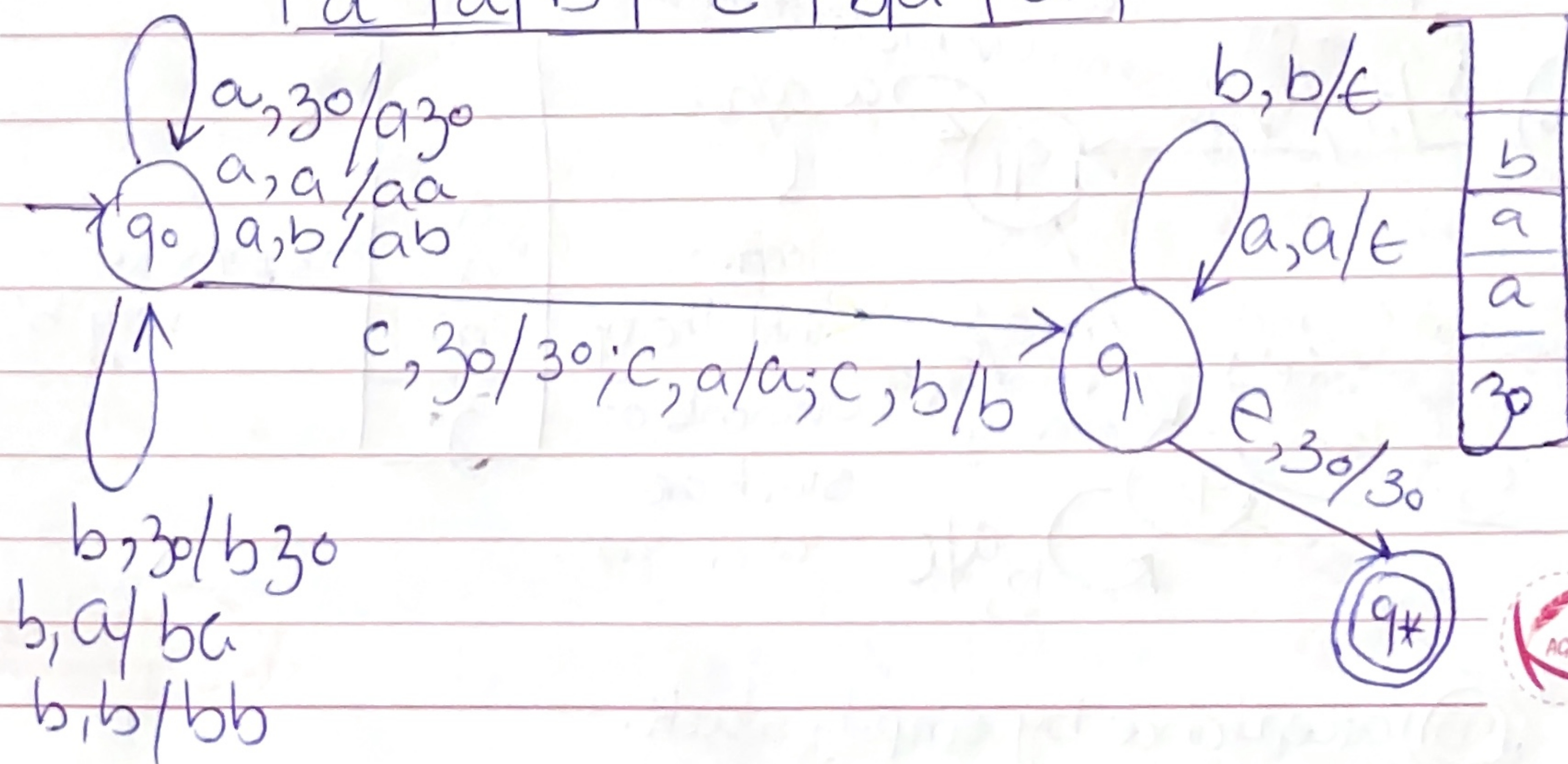
here to cater multiple 'a's that corresponding with 2x multiples of b.

Question: Design a PDA for $wcwr \mid w \in (a,b)^*$?

let

a a b c b a a

start palindrome.

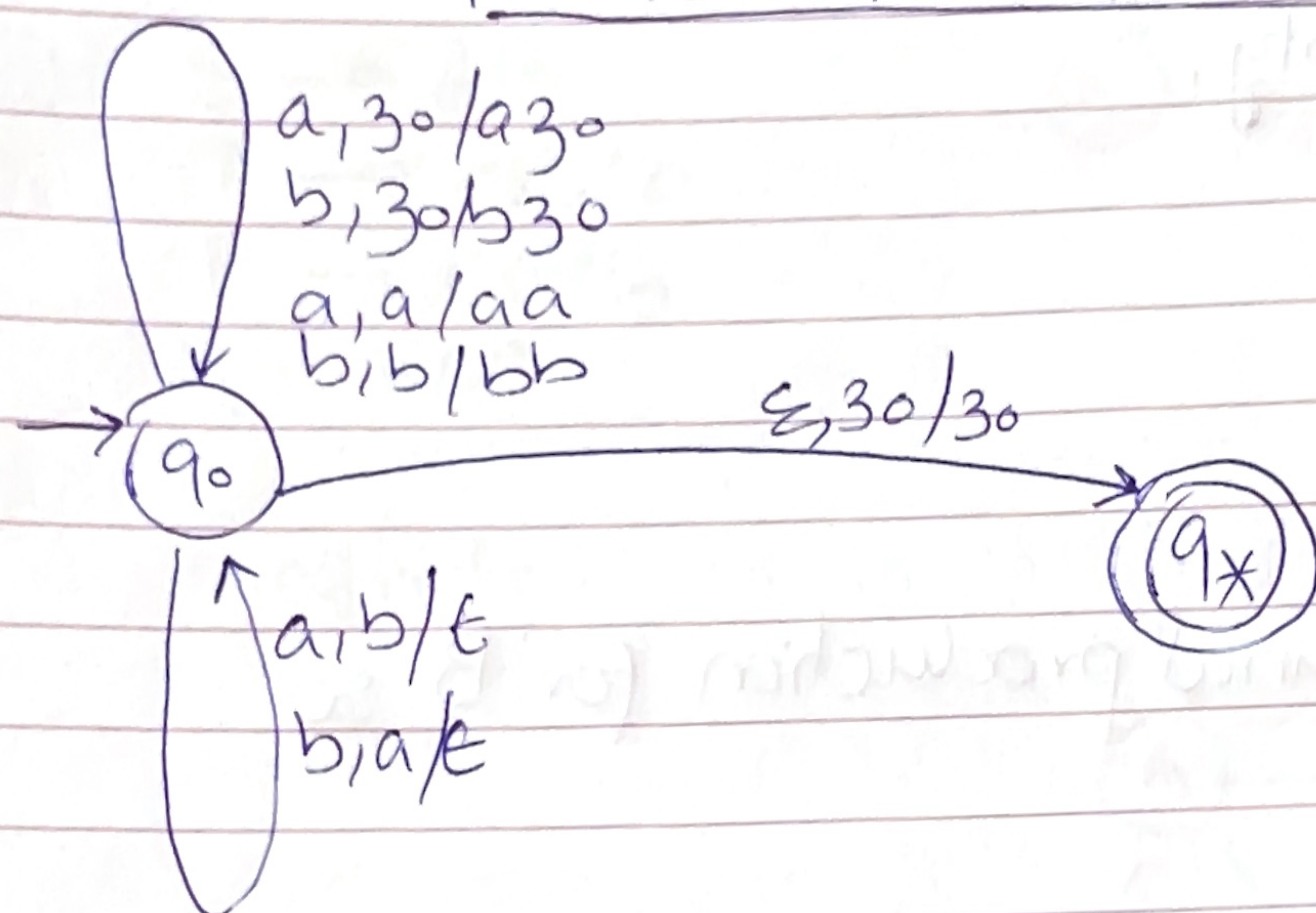


Question:- Construct A PDA that accepts
 $L = \{ |W|a=b \mid W \in (a,b)^* \}$

↓
 a string that has equal no. of a's
 and b's.

let a tape

a | b | b | b | a | b | b | a | a | a | ε



Note:

The power of non deterministic PDA &
 Deterministic PDA is not the same.

⇒ NDPDA is more powerful, because it can
 move to more than one state.

→ maybe there exists a grammar that cannot
 be made through DPDA but can be made
 through NDPDA.

Question: Consider the following CFG and identify
 which of the following CFG generates
 Empty language?

$$\textcircled{i} \quad S \rightarrow aAB / Aa$$

$$A \rightarrow a$$

lets minimize our grammar.

$\textcircled{i}$ if you remove the production of aAB as B as a nonterminal production is not given still $A \rightarrow a$ can lead to some terminal.

$\therefore$ Non empty.

$$\textcircled{ii} \quad S \rightarrow aAB$$

$$A \rightarrow a/b$$

There is no terminal production for 'B' so this empty.

$$\textcircled{iii} \quad S \rightarrow aAB / aB$$

$A \rightarrow aBb$
 $B \rightarrow aA$ } these two productions are sending over one another but not to a terminal.

$\therefore$ empty!

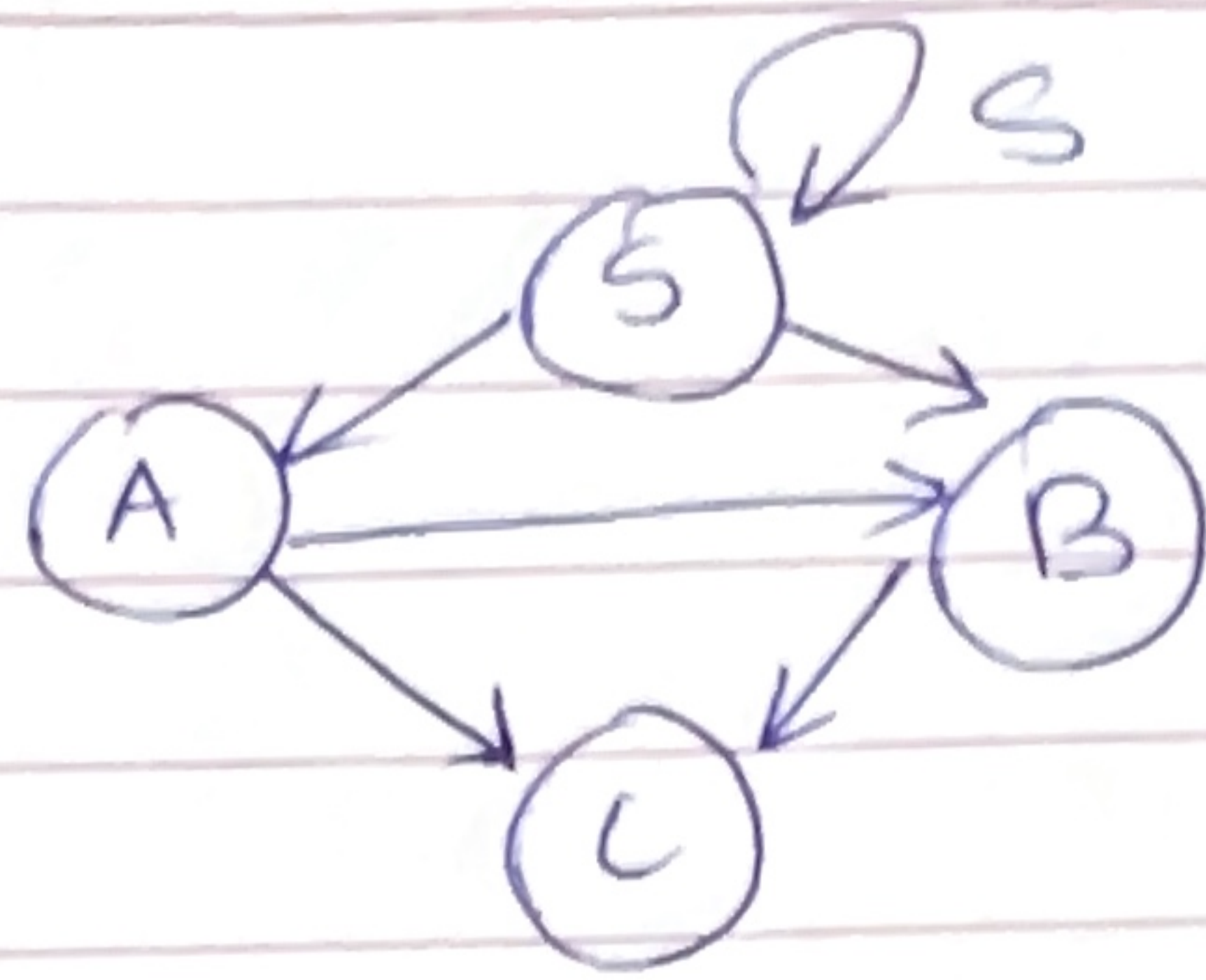
Question: Consider the following CFG and identify which of the following CFG generate finite.

$\hookrightarrow$ identified through recursion.

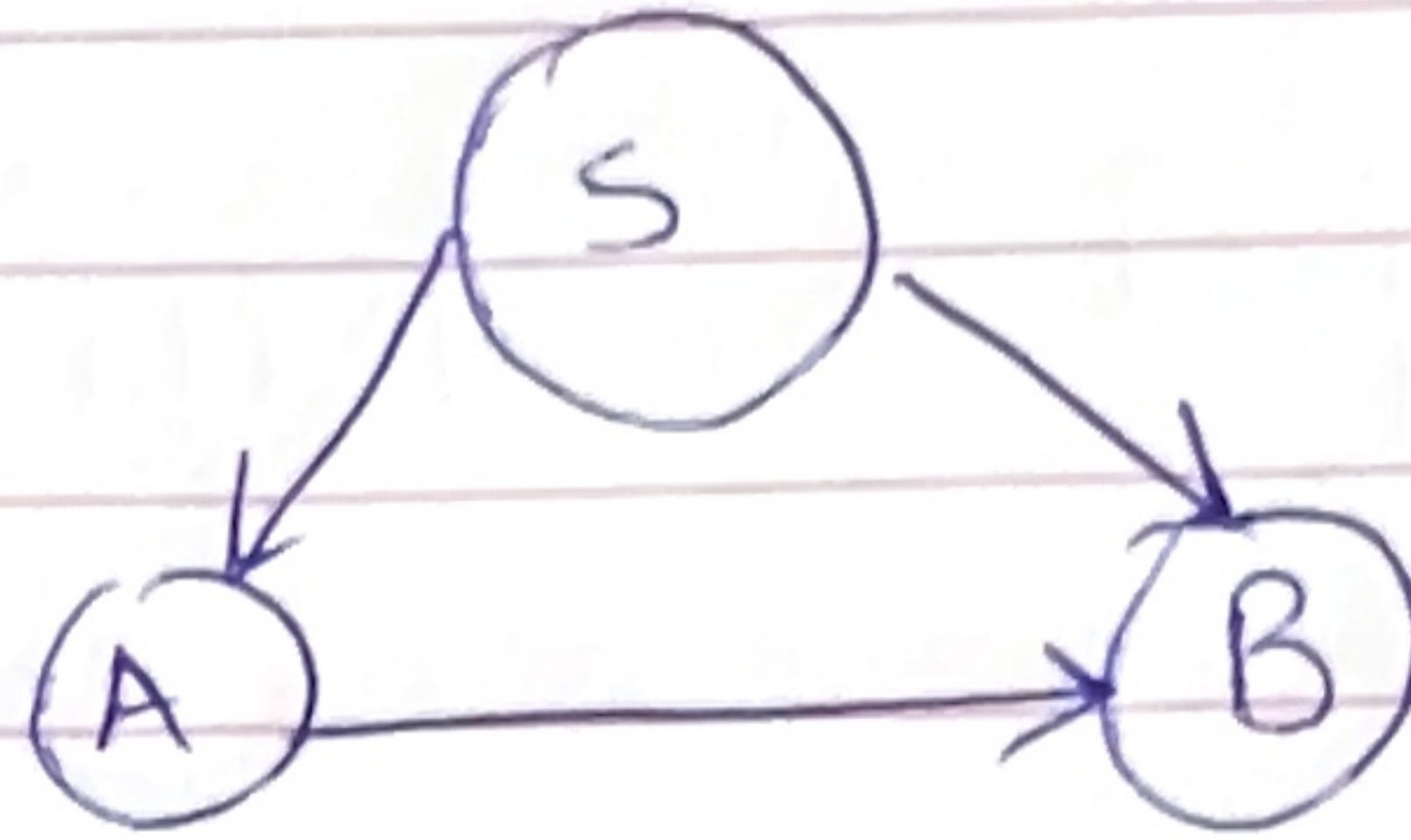
Step no. 1: Draw vertices for nonterminals.

Step no. 2: Draw a self loop ^{for cycles} which shows infiniteness due to recursion.

- ① $S \rightarrow SS / AB$
 $A \rightarrow BC / a$
 $B \rightarrow CC / b$
 infinite.



- ② $S \rightarrow AB$
 $A \rightarrow B / a$
 finite.



- ③ $S \rightarrow AB$
 $A \rightarrow BC / a$
 $B \rightarrow CC / b$
 $C \rightarrow AB$

infinite.

